

Data Architecture for AI · MSc 2025-2026

Comprendre le projet Labo Santé

De zéro à une architecture de données containerisée
tout expliqué, pas à pas, sans pré-requis

Cours pédagogique complet · Aline Maire · 9 juin 2026

Repo : github.com/Almaire-Lab/lab0-sante

Sommaire

1. Partie 1 — Les fondamentaux (à partir de zéro)

1. Pourquoi a-t-on besoin d'une base de données ?
2. SQL vs NoSQL — c'est quoi la différence ?
3. Docker en 10 minutes (image, conteneur, volume, réseau)
4. Docker Compose — orchestrer plusieurs services

2. Partie 2 — Le projet Labo Santé

1. Le problème métier
2. La solution : 5 conteneurs qui collaborent
3. Vue d'ensemble de l'architecture

3. Partie 3 — Décryptage des livrables

1. Cahier des charges
2. Schéma SQL — pourquoi ces 5 tables ?
3. Schéma NoSQL — pourquoi un document ?
4. Diagramme des services

4. Partie 4 — Décryptage du code

1. `schema.sql` — la DDL ligne par ligne
2. `docker-compose.yml` — service par service
3. `.env` et `.gitignore` — gestion des secrets

5. Partie 5 — La stack en action

1. Cycle de vie d'un conteneur
2. Comment les conteneurs se parlent
3. Les 3 interfaces web (pgAdmin, Mongo Express, Portainer)
4. Le rôle de chaque healthcheck

6. Partie 6 — Pour aller plus loin

1. Ce qui arrive en Session 2
2. FAQ — toutes les questions « pourquoi ? »
3. Glossaire
4. Commandes utiles

Partie 1

Les fondamentaux

1.1 Pourquoi a-t-on besoin d'une base de données ?

Imagine que tu écris la liste de tes patients dans un fichier Excel. Tout va bien tant que :

- tu es **seul** à éditer le fichier,
- tu as **quelques centaines** de lignes,
- tu ne fais que de la **lecture/écriture simple**.

Dès qu'on dépasse ces 3 conditions, Excel ne suffit plus. Une **base de données (BDD)** apporte 4 super-pouvoirs :

Super-pouvoir	Pourquoi c'est crucial
Accès concurrent	Plusieurs personnes/programmes peuvent lire et écrire en même temps sans corrompre les données.
Intégrité	La BDD refuse les données incohérentes (ex: une prescription pour un patient qui n'existe pas).
Performance	Avec des index, retrouver 1 ligne parmi 10 millions prend < 1 ms.
Durabilité (ACID)	Une transaction commitée n'est jamais perdue, même en cas de coupure de courant.



Le mot ACID

A=Atomicité, C=Cohérence, I=Isolation, D=Durabilité. C'est le contrat qu'une BDD relationnelle (comme PostgreSQL) te garantit pour chaque transaction. NoSQL est souvent plus souple.

1.2 SQL vs NoSQL — c'est quoi la différence ?

SQL SQL — la base relationnelle classique

On modélise les données en **tables** avec un schéma rigide (colonnes typées). On relie les tables avec des **clés étrangères**. On interroge avec le langage **SQL** (Structured Query Language).

Exemple : pour stocker un patient et ses prescriptions, on aura :

```
patients(id, nom, prenom, date_naissance, ...)
prescriptions(id, patient_id, medecin_id, date, ...)
                ↑           ↑
            FK vers patients  FK vers medecins
```

Champions du SQL : PostgreSQL (notre choix), MySQL, MariaDB, SQL Server, Oracle.

NoSQL NoSQL — schéma libre

Plusieurs familles : document (MongoDB, notre choix), clé-valeur (Redis), colonnes (Cassandra), graphe (Neo4j). Le plus courant aujourd'hui : **document**. On stocke des documents JSON sans schéma fixe.

```
{ "_id": "abc", "patient_id": 1428,
  "symptoms": [ {"label": "toux", "duree": 10}, {"label": "fièvre"} ],
  "diagnosis": { "primary": {"code": "J20.9"}, "severity": "moderee" }
}
```

Quand choisir quoi ?

	SQL — PostgreSQL	NoSQL — MongoDB
Schéma	fixe, défini à l'avance	libre, évolue par doc
Relations	fortes (JOIN, FK)	faibles (par référence)
Données variables	compliqué	natif
Transactions ACID	oui, complet	oui depuis MongoDB 4.0 (multi-doc)
Quand l'utiliser	référentiels, comptabilité, prescriptions	logs, contenus libres, consultations médicales



Dans notre projet

On utilise **les deux** : SQL pour ce qui est structuré et critique (patients, médicaments, prescriptions) et NoSQL pour ce qui est variable (consultations avec symptômes libres et diagnostics structurés).

1.3 Docker en 10 minutes

Docker, c'est de la **virtualisation légère**. Au lieu de lancer une vraie VM (lourde, 1 OS complet), on lance un **conteneur** : un processus isolé avec ses propres fichiers et ses propres ports.

Les 4 concepts à retenir

Concept	Définition simple	Analogie
IMAGE	un "modèle" figé contenant l'application + ses dépendances (ex: <code>postgres:15-alpine</code>)	un fichier ISO de jeu vidéo
CONTENEUR	une instance qui tourne à partir d'une image	le jeu installé qui s'exécute
VOLUME	un dossier persistant en dehors du conteneur — survit aux redémarrages	tes sauvegardes sur le disque dur
RÉSEAU	un réseau virtuel privé entre conteneurs	un LAN entre tes machines

⚠ Sans volume → perte de données

Quand on fait `docker rm` un conteneur, tout ce qui était à l'intérieur disparaît. Si Postgres écrit ses données dans `/var/lib/postgresql/data` sans volume monté, on perd tout au prochain `down` .

D'où le volume `pgdata` dans notre projet.

Le mini-vocabulaire

Commande	Rôle
<code>docker pull postgres:15</code>	télécharger l'image Postgres v15
<code>docker run postgres:15</code>	démarrer un nouveau conteneur
<code>docker ps</code>	lister les conteneurs en marche
<code>docker logs <nom></code>	voir la sortie standard d'un conteneur
<code>docker exec -it <nom> bash</code>	entrer dans un conteneur (shell interactif)
<code>docker stop <nom></code>	arrêter proprement
<code>docker rm <nom></code>	supprimer le conteneur (data perdue si pas de volume)

1.4 Docker Compose — orchestrer plusieurs services

Avec `docker run`, lancer 5 conteneurs liés (postgres + pgadmin + mongo + mongo-express + portainer) prendrait 5 commandes longues, qu'il faudrait répéter à chaque démarrage. **Docker Compose** règle ça : on décrit tout dans un seul fichier YAML (`docker-compose.yml`), et une seule commande lance/arrête le tout.

```
docker compose up -d      # démarre toute la stack en arrière-plan
docker compose ps         # voit l'état de chaque service
docker compose logs -f    # suit les logs en direct
docker compose down       # arrête tout (volumes préservés)
docker compose down -v    # arrête tout ET supprime les volumes (⚠ perte de données)
```

Anatomie d'un `docker-compose.yml`

```
services:                # liste des conteneurs à créer
  monservice:
    image: ...            # image source
    ports: ...            # mapping de ports hôte:conteneur
    volumes: ...          # disques persistants
    environment: ...      # variables d'environnement
    depends_on: ...       # ordre de démarrage
    healthcheck: ...       # test de santé périodique

volumes: ...              # volumes nommés partagés
networks: ...             # réseaux virtuels
```

💡 Pourquoi YAML ?

YAML est un format texte basé sur l'indentation, lisible par un humain. **Attention aux espaces** : 2 espaces partout, jamais de tabulation. Sinon Docker refuse le fichier.

Partie 2

Le projet Labo Santé

2.1 Le problème métier

« Un laboratoire de santé veut centraliser ses données médicales aujourd'hui éclatées entre plusieurs logiciels et fichiers Excel. À terme, ces données alimenteront des modèles IA. »

Concrètement, on a 5 types de données à gérer :

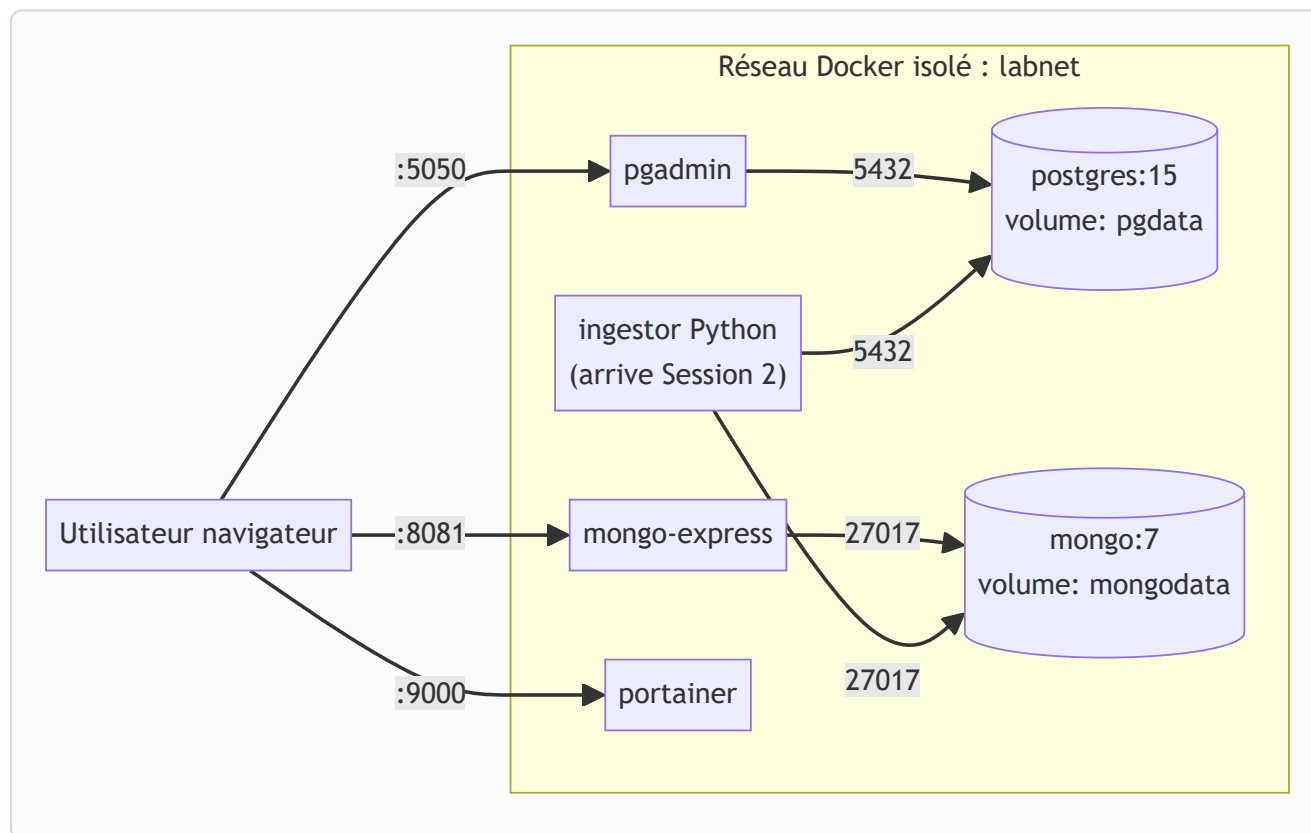
Donnée	Exemple concret	Volume estimé
Médecins	Dr Dupont, cardiologue	~ 100
Patients	Mme Martin, née le 12/04/1985	~ 100 000
Médicaments	Doliprane 500 mg, comprimé	~ 5 000
Prescriptions	Le 9 juin, Dr Dupont prescrit Doliprane à Mme Martin pour 5 jours	~ 10 M/an
Consultations	« Toux depuis 10 jours, fièvre 38.4°C, diagnostic : bronchite aiguë »	~ 6 M/an

2.2 La solution : 5 conteneurs qui collaborent

Plutôt que de tout installer manuellement sur le PC, on emballe chaque brique dans un conteneur Docker. Résultat : **une seule commande** (`docker compose up -d`) déploie tout, partout, à l'identique.

Conteneur	Rôle	Pourquoi cette image
<code>postgres</code>	la BDD relationnelle (les 4 référentiels + prescriptions)	la référence open-source SQL · version 15 stable
<code>mongo</code>	la BDD documentaire (les consultations)	le standard du document JSON · version 7
<code>pgadmin</code>	interface web pour Postgres	évite d'apprendre <code>psql</code> en CLI
<code>mongo-express</code>	interface web pour Mongo	idem pour Mongo
<code>portainer</code>	tableau de bord Docker (logs, CPU, RAM...)	monitoring sans devoir taper <code>docker stats</code>

2.3 Vue d'ensemble de l'architecture



3 idées clés à comprendre :

1. **Tous les conteneurs sont dans le même réseau Docker (**labnet**)**. Ils s'appellent par leur **nom de service** (**postgres** , **mongo**) — pas par **localhost** ni par IP.
2. **Seuls 3 ports sont ouverts sur l'hôte** (5050, 8081, 9000), uniquement sur **127.0.0.1** — donc inaccessibles depuis l'extérieur de ta machine.
3. **Les 2 BDD ont chacune un volume nommé**. Tes données survivent à **docker compose down** .

Partie 3

Décryptage des livrables

3.1 Le cahier des charges

Rôle : mettre par écrit ce que la plateforme *doit* faire **avant** d'écrire la moindre ligne de code. Sans cahier des charges, on développe au pifomètre et on rate la cible.

Structure d'un bon cahier des charges

Section	Question à laquelle elle répond
1. Contexte	Pourquoi ce projet existe ?
2. Données à gérer	Qu'est-ce qu'on stocke et où ?
3. Besoins fonctionnels (F1...F6)	Que doit faire l'utilisateur final ?
4. Besoins techniques	Quelles contraintes d'infrastructure ?
5. Périmètre	Qu'est-ce qui est hors du projet ?
6. Critères d'acceptation	Comment on sait que c'est fini ?

Le piège classique

Oublier le périmètre. Sans dire « hors projet : authentification utilisateur, RGPD, frontal métier », on se retrouve à tout faire et rien ne marche.

3.2 Schéma SQL — pourquoi ces 5 tables ?

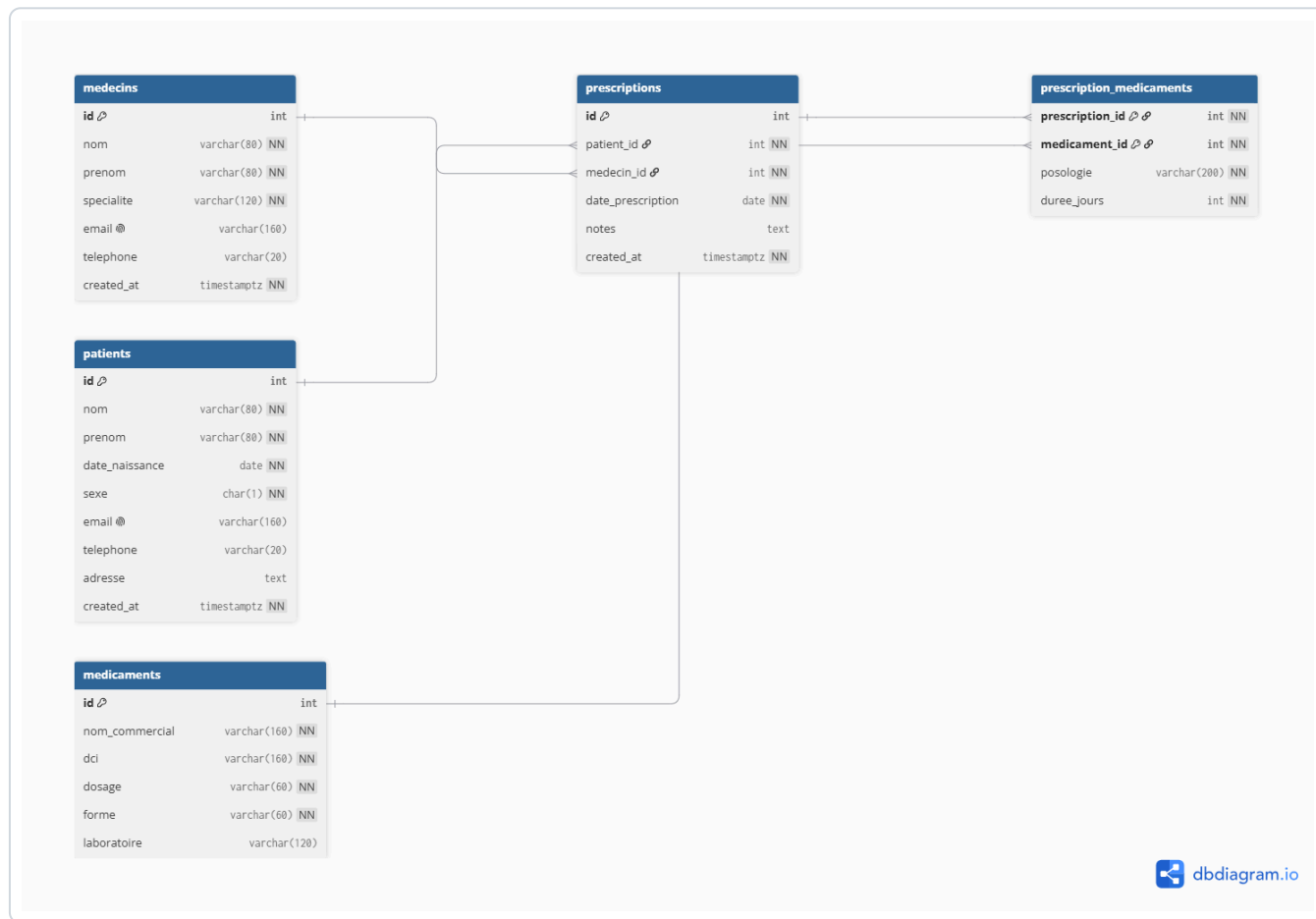


Schéma exporté de dbdiagram.io

Le raisonnement

On part des **entités du monde réel** : médecin, patient, médicament, prescription. Chaque entité devient une table. Ensuite, on identifie les **relations** :

- Une **prescription** est faite **par 1 médecin** pour **1 patient** → 2 clés étrangères (`medecin_id` , `patient_id`).
- Une prescription contient **1 ou plusieurs médicaments**, et un médicament peut figurer dans **plusieurs prescriptions**. C'est une relation **N-N** (many-to-many).

💡 La règle d'or des N-N

On ne peut pas modéliser une relation N-N avec 2 tables. Il faut **une table intermédiaire** (ici `prescription_medecaments`) avec une clé primaire composite (`prescription_id` , `medicament_id`) qui empêche le même médicament d'apparaître 2× dans la même prescription.

Les contraintes — la BDD nous protège

Type	Exemple dans notre schéma	Ce que ça interdit
PRIMARY KEY	id SERIAL PRIMARY KEY	2 médecins avec le même id
FOREIGN KEY	patient_id REFERENCES patients(id)	une prescription pour un patient fantôme
UNIQUE	email VARCHAR(160) UNIQUE	2 médecins avec le même email
CHECK	CHECK (sexe IN ('M','F','X'))	un sexe autre que M/F/X
NOT NULL	nom VARCHAR(80) NOT NULL	un patient sans nom
ON DELETE	ON DELETE RESTRICT	supprimer un patient ayant des prescriptions

Les index — la performance

Sans index, retrouver toutes les prescriptions d'un patient parmi 10 millions = scan complet de la table (lent). Avec un index sur `patient_id` = recherche en arbre B-tree (rapide). On indexe ce qui sera **filtré ou trié** souvent.

3.3 Schéma NoSQL — pourquoi un document ?

Une consultation médicale, c'est :

- quelques métadonnées fixes (patient, médecin, date),
- une **liste de symptômes** de longueur variable, avec des attributs différents par symptôme (température pour la fièvre, durée pour la toux...),
- un **diagnostic** qui peut avoir plusieurs codes secondaires,
- éventuellement des **pièces jointes** (audio d'auscultation, photo...).

En SQL, modéliser ça proprement demanderait 5+ tables (`consultations` , `symptomes` , `diagnostics_primaire` , `diagnostics_secondaires` , `attachments` ...) avec des jointures complexes à **chaque lecture**. C'est lourd et lent.

En NoSQL document, tout tient dans **un seul JSON** :

```
{
  "patient_id": 1428,      ← lien logique vers PostgreSQL
  "medecin_id": 12,
  "date": "2026-06-09T09:30:00Z",
  "symptoms": [           ← longueur variable, structure libre
    { "label": "toux", "duree_jours": 10 },
    { "label": "fièvre", "temperature_c": 38.4 }
  ],
  "diagnosis": {
    "primary": { "code_cim10": "J20.9", "label": "Bronchite aiguë" },
    "secondary": [ { "code_cim10": "R05", "label": "Toux" } ]
  }
}
```

💡 Le pattern "polyglot persistence"

Utiliser **plusieurs types de BDD selon le besoin** dans le même projet. On parle de "polyglot persistence". C'est exactement ce qu'on fait ici : Postgres pour le structuré + Mongo pour le semi-structuré, reliés par `patient_id`.

Index recommandés

- `{ patient_id: 1 }` — pour récupérer toutes les consultations d'un patient (cas usage le + fréquent)
- `{ date: -1 }` — chronologie inverse (les plus récentes d'abord)
- `{ "diagnosis.primary.code_cim10": 1 }` — recherche statistique par diagnostic

3.4 Diagramme des services

Le diagramme d'architecture sert à **visualiser** qui parle à qui. C'est le premier truc que regarde un nouveau dev (ou ton prof) avant d'ouvrir le code.

3 conventions importantes :

1. Les **flèches** représentent une connexion réseau (TCP). On note le port à côté.
2. Les **cylindres** représentent des BDD (avec leur volume).
3. Le **cadre** **labnet** représente l'isolation réseau Docker.

Partie 4

Décryptage du code

4.1 `schema.sql` — la DDL ligne par ligne

DDL = *Data Definition Language* : les ordres SQL qui **créent la structure** (tables, contraintes, index). À distinguer du DML (INSERT/UPDATE/DELETE) qui manipule les données.

`sql/schema.sql` – exécuté UNE fois au 1er démarrage

```
CREATE TABLE IF NOT EXISTS medecins (  
  id          SERIAL PRIMARY KEY,  
  nom         VARCHAR(80) NOT NULL,  
  prenom      VARCHAR(80) NOT NULL,  
  specialite  VARCHAR(120) NOT NULL,  
  email       VARCHAR(160) UNIQUE,  
  telephone  VARCHAR(20),  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Mot-clé	Explication
<code>CREATE TABLE IF NOT EXISTS</code>	crée la table si elle n'existe pas déjà (idempotent — peut être ré-exécuté sans erreur)
<code>SERIAL PRIMARY KEY</code>	entier auto-incrémenté, clé primaire (= UNIQUE + NOT NULL + index automatique)
<code>VARCHAR(80)</code>	chaîne de longueur variable, max 80 caractères
<code>NOT NULL</code>	la colonne ne peut pas être vide
<code>UNIQUE</code>	aucun doublon possible dans cette colonne
<code>TIMESTAMPTZ</code>	date+heure avec fuseau horaire
<code>DEFAULT NOW()</code>	si non fournie, prend la date courante

Le truc magique : auto-exécution au boot

`docker-compose.yml` – montage du fichier SQL

```
postgres:  
  image: postgres:15-alpine  
  volumes:  
    - pgdata:/var/lib/postgresql/data  
    - ./sql/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
```

L'image officielle Postgres **exécute automatiquement** tout fichier `.sql` trouvé dans `/docker-entrypoint-initdb.d/` **au premier démarrage uniquement** (quand le volume `pgdata` est vide). Le `:ro` = *read-only* côté conteneur.

 **Une seule fois !**

Si tu changes le schéma après le 1er démarrage, le fichier ne sera **pas ré-exécuté**. Il faut soit faire un `docker compose down -v` (perd toutes les données), soit appliquer une migration manuelle via `psql` ou pgAdmin.

4.2 `docker-compose.yml` — service par service

Service `postgres`

```
postgres:
  image: postgres:15-alpine          # image officielle, version 15, légère
  container_name: labo-postgres      # nom personnalisé du conteneur
  restart: unless-stopped            # redémarre auto sauf si arrêt manuel
  environment:
    POSTGRES_USER:   ${POSTGRES_USER}    # lu depuis le .env
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    POSTGRES_DB:     ${POSTGRES_DB}
  ports:
    - "127.0.0.1:5432:5432"             # expose 5432 SEULEMENT en local
  volumes:
    - pgdata:/var/lib/postgresql/data    # persistance
    - ./sql/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER"]
    interval: 5s
    retries: 10
  networks: [labnet]
```

Clé	Rôle
<code>image</code>	quelle image Docker utiliser (téléchargée depuis Docker Hub si absente)
<code>environment</code>	variables d'env à injecter dans le conteneur
<code>\${VAR}</code>	substitution : lit la valeur depuis le fichier <code>.env</code>
<code>ports</code>	<code>hôte:conteneur</code> — <code>127.0.0.1:</code> limite l'accès à la machine locale
<code>volumes</code>	monte un volume nommé OU un fichier/dossier local
<code>healthcheck</code>	commande exécutée régulièrement pour savoir si le service est <i>healthy</i>
<code>networks</code>	réseau(x) auquel le service appartient

Service `pgadmin` avec `depends_on`

```
pgadmin:
  image: dpage/pgadmin4:latest
  ports:
    - "127.0.0.1:5050:80"
  depends_on:
    postgres:
      condition: service_healthy
```

 Pourquoi `service_healthy` et pas juste `depends_on` ?

`depends_on` tout court attend que le **conteneur démarre**. Mais Postgres peut être démarré sans être **prêt à recevoir des connexions** (5–10 sec d'init). Avec `condition: service_healthy`, Compose attend que le healthcheck retourne OK. Sinon pgAdmin essaie de se connecter, échoue, redémarre, etc.

4.3 `.env` et `.gitignore` — la sécurité

Règle d'or : aucun mot de passe en clair dans le code versionné. On utilise 3 fichiers :

Fichier	Commité sur Git ?	Rôle
<code>.env</code>	✗ JAMAIS	les vrais secrets, propre à chaque machine
<code>.env.example</code>	✓ oui	modèle avec des valeurs bidon, documente les variables nécessaires
<code>.gitignore</code>	✓ oui	liste de patterns à ignorer — contient <code>.env</code>



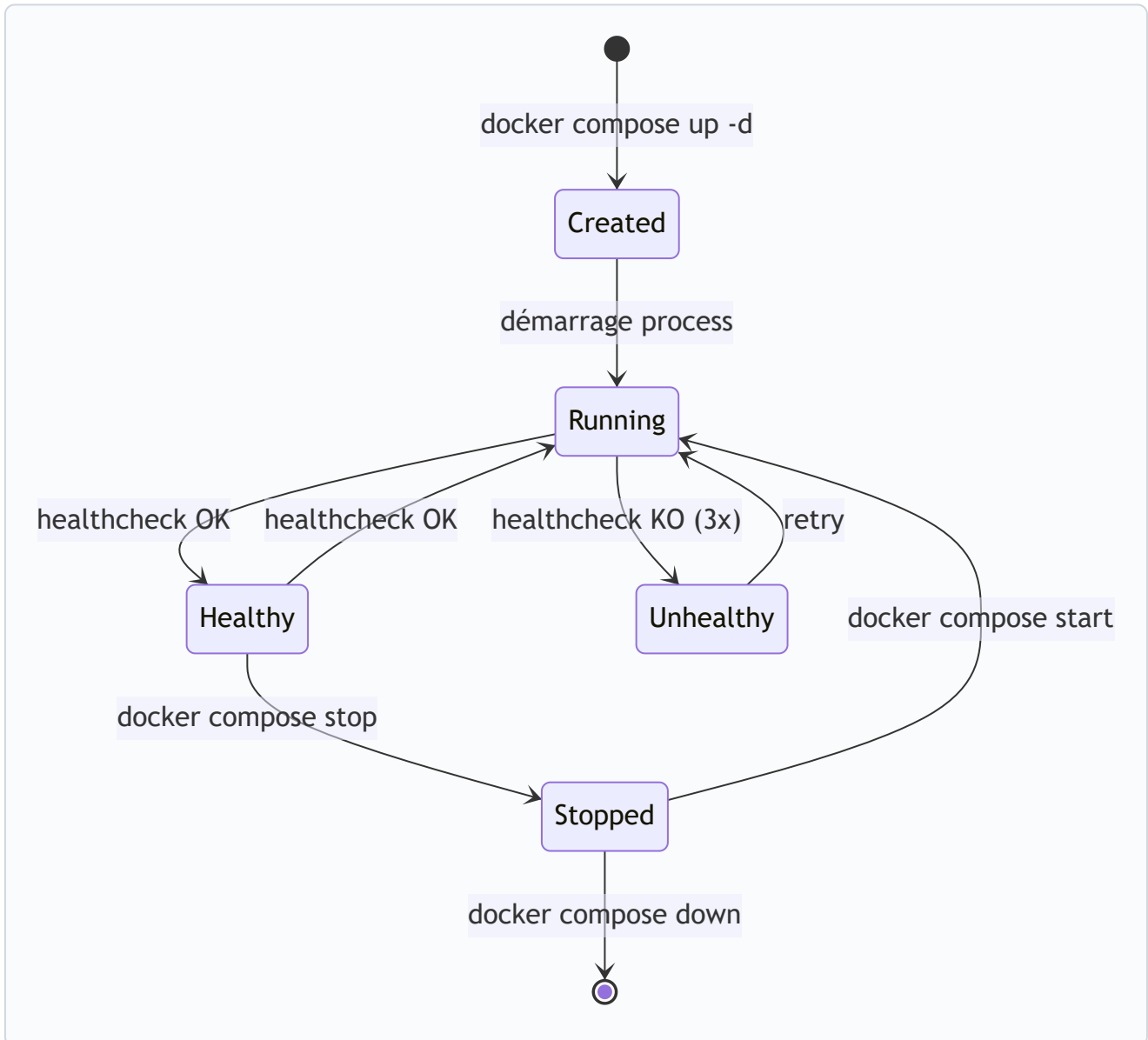
Anti-pattern à connaître

Push d'un `.env` avec des vrais mots de passe sur GitHub public = **compromis en quelques minutes**. GitHub a même un scanner automatique (Secret Scanning) qui détecte les clés AWS/Azure/etc. Si ça arrive : changer immédiatement les mots de passe (le revert Git ne suffit pas, l'historique reste).

Partie 5

La stack en action

5.1 Cycle de vie d'un conteneur



5.2 Comment les conteneurs se parlent

Docker crée un mini-DNS interne dans chaque réseau. Le nom DNS d'un service = son nom dans `docker-compose.yml`. Donc depuis n'importe quel conteneur de `labnet`, on peut faire :

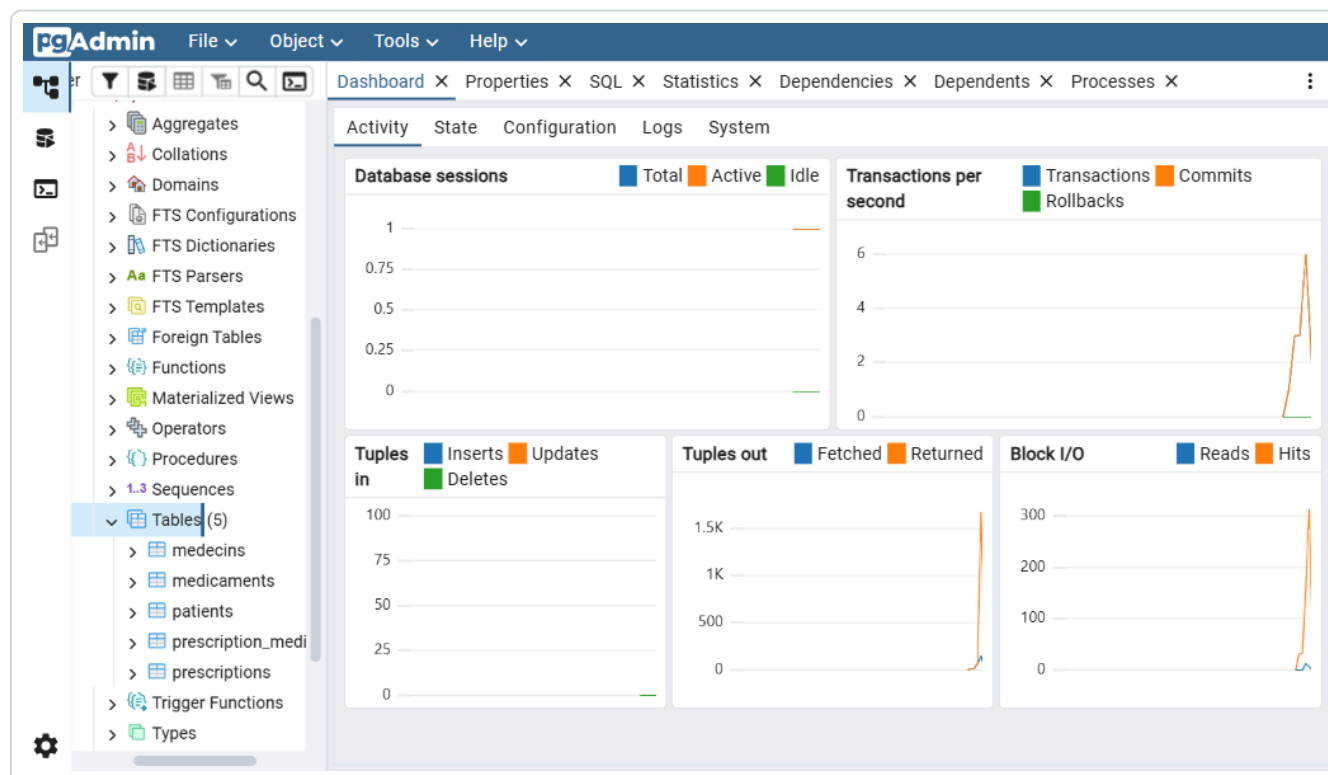
```
# Depuis pgadmin, pour atteindre Postgres :  
host = "postgres"      # ← pas "localhost", pas une IP !  
port = 5432  
  
# Depuis l'ingestor (Session 2), pour atteindre Mongo :  
mongodb://lab_admin:password@mongo:27017/labo
```

💡 Pourquoi pas `localhost` ?

Dans un conteneur, `localhost` = le conteneur lui-même. Pas la machine hôte. Pas un autre conteneur. Donc `localhost:5432` depuis pgadmin essaierait de se connecter à pgadmin lui-même — ça ne marche pas.

5.3 Les 3 interfaces web

pgAdmin (port 5050) — explorer PostgreSQL



Arbre Servers → labo → Databases → labo → Schemas → public → Tables (5)

Ce que tu peux y faire :

- Voir les tables, leurs colonnes, leurs contraintes
- Lancer des requêtes SQL ad-hoc (Query Tool)
- Voir les données ligne par ligne
- Backup / Restore

Mongo Express (port 8081) — explorer MongoDB

Plus simple que pgAdmin. Liste les **databases** → **collections** → **documents**. On peut ajouter/modifier/supprimer un document, créer des index, exporter en JSON/CSV.

Portainer (port 9000) — observer Docker

PORTAINER.io

COMMUNITY EDITION

Home

local

Dashboard

Templates

Stacks

Containers

Images

Networks

Volumes

Events

Host

Administration

Portainer Community Edition 2.39.3 LTS

Containers

Container list

Containers

Search...

Start

Stop

Kill

Restart

Pause

Resume

☐	Name	State	Filter	Quick Actions	Stack	Image
☐	labo-mongo	healthy			labo-sante	mongo:7
☐	labo-mongo-express	running			labo-sante	mongo-express:1
☐	labo-pgadmin	running			labo-sante	dpage/pgadmin4:latest
☐	labo-portainer	running			labo-sante	portainer/portainer-ce:latest
☐	labo-postgres	healthy			labo-sante	postgres:15-alpine

Items per page 10

Vue d'ensemble : 5 conteneurs, statuts, métriques

Portainer n'est PAS lié à Postgres ou Mongo : c'est un tableau de bord **de Docker lui-même**. Il se branche sur `/var/run/docker.sock` (le socket de l'API Docker locale) pour voir tous les conteneurs en cours, leurs logs en direct, leur conso CPU/RAM, etc.

5.4 Le rôle de chaque healthcheck

Un healthcheck = une commande exécutée toutes les X secondes **dans** le conteneur. Si elle réussit, le conteneur passe en état *healthy*. Sinon (après N retries), il bascule en *unhealthy* et Docker peut le redémarrer.

Service	Commande de healthcheck	Ce qu'elle vérifie
postgres	<code>pg_isready -U \$POSTGRES_USER</code>	le serveur Postgres répond aux connexions
mongo	<code>mongosh --eval "db.adminCommand('ping')"</code>	le serveur Mongo répond à un ping admin

Partie 6

Pour aller plus loin

6.1 Ce qui arrive en Session 2

Session 1 a posé l'infra. Session 2 va y faire **couler des données**.

Le nouveau service : `ingestor` Python

Un conteneur Python qui :

1. **Génère** des données synthétiques (faker pour les noms, dates aléatoires...)
2. **Insère en masse** dans Postgres via `psycopg2` (driver Python pour Postgres)
3. **Insère en masse** dans Mongo via `pymongo` (driver Python pour Mongo)
4. **Mesure** le temps total et la RAM utilisée

On va tester 3 paliers : **60 000**, **600 000**, **6 000 000** de lignes pour voir comment ça scale.

Notions nouvelles à venir

Notion	De quoi il s'agit
Dockerfile	recette pour construire <i>notre propre</i> image (Python + dépendances)
<code>requirements.txt</code>	liste des packages Python (<code>pip install -r</code>)
Idempotence	relancer l'ingestion ne doit pas créer de doublons
Bulk insert	insérer 10 000 lignes en 1 requête (vs 10 000 requêtes individuelles)
Transactions	grouper des écritures pour qu'elles soient atomiques

6.2 FAQ — les "pourquoi ?"

Pourquoi PostgreSQL plutôt que MySQL ?

PostgreSQL est plus rigoureux (typage strict, contraintes plus puissantes), supporte le JSON nativement (`JSONB`), et reste 100 % open-source. C'est le choix « par défaut » dans le monde moderne (Heroku, Supabase, Cloud SQL...).

Pourquoi MongoDB plutôt qu'un champ JSONB dans Postgres ?

Bonne question — on aurait pu ! Mais MongoDB est plus performant sur le **requêtage profond** dans les documents (ex: `diagnosis.primary.code_cim10`) et ses index sur les sous-champs sont plus simples. Et pédagogiquement, le cours veut nous faire manipuler les deux mondes.

Pourquoi ne pas exposer Postgres sur 0.0.0.0 ?

`0.0.0.0:5432` ouvrirait le port à **tout l'Internet** si la machine est exposée. `127.0.0.1:5432` n'ouvre que pour la machine locale. C'est le minimum sécurité.

Pourquoi `postgres:15-alpine` et pas `postgres:15` ?

La variante *alpine* est basée sur Alpine Linux, ~5× plus légère (60 MB vs 300 MB). Démarrage + téléchargement + scan de sécurité plus rapides.

Pourquoi pgAdmin demande un master password ?

pgAdmin chiffre les mots de passe des connexions qu'il sauvegarde (le mot de passe Postgres notamment) avec ton master password. Il est local à pgAdmin, pas lié à Postgres.

Pourquoi `:ro` sur le volume du `schema.sql` ?

Read-only : le conteneur ne peut pas écrire sur ton fichier hôte. Bonne pratique systématique pour les fichiers de configuration montés.

Pourquoi pgAdmin a refusé `admin@labo.local` ?

Le TLD `.local` est **réservé par l'IETF** (RFC 6762, mDNS). pgAdmin valide strictement les emails et refuse les domaines réservés. `example.com` (réservé doc) ou un vrai domaine fonctionnent.

Pourquoi Portainer s'est verrouillé après quelques minutes ?

Sécurité anti-hijack : si personne ne crée le compte admin dans les 5 min après le 1er boot, Portainer se verrouille pour empêcher un attaquant qui découvrirait le port 9000 de prendre la main. Un redémarrage du conteneur réinitialise le timer.

Que se passe-t-il si je `docker compose down -v` ?

Le **-v** supprime aussi les **volumes**. Donc **pgdata** , **mongodata** , **pgadmin** , **portainer** = perte totale des données. **down** sans **-v** garde les volumes : tu peux re-up et tout est encore là.

6.3 Glossaire

Terme	Définition courte
ACID	Atomicité, Cohérence, Isolation, Durabilité — garanties d'une transaction SQL
CIM-10	Classification Internationale des Maladies, version 10. "J20.9" = bronchite aiguë.
Conteneur	processus isolé créé à partir d'une image Docker
DDL / DML	Data Definition Language (CREATE/ALTER) vs Data Manipulation Language (INSERT/UPDATE)
Dockerfile	fichier décrivant comment construire une image Docker (vu en Session 2)
Foreign Key (FK)	colonne qui pointe vers la PK d'une autre table
Healthcheck	commande périodique qui dit si un conteneur est <i>healthy</i>
Idempotence	propriété d'une opération qu'on peut ré-exécuter sans effet de bord
Image	"modèle" figé contenant l'app + dépendances
Index	structure auxiliaire qui accélère les recherches sur une colonne
JSON / JSONB	format de document. JSONB = stockage binaire optimisé (PostgreSQL)
N-N (many-to-many)	relation entre 2 tables nécessitant une table de liaison
Primary Key (PK)	identifiant unique d'une ligne
Réseau Docker	LAN virtuel privé entre conteneurs
Schema (SQL)	conteneur logique de tables dans une BDD Postgres (souvent <code>public</code> par défaut)
Volume	stockage persistant Docker, survit aux redémarrages
YAML	format texte basé sur l'indentation (utilisé pour docker-compose)

6.4 Commandes utiles à connaître par cœur

Docker / Compose

```
# Démarrer toute la stack en arrière-plan
docker compose up -d

# Voir l'état des services
docker compose ps

# Suivre les logs de tous les services
docker compose logs -f

# Logs d'un seul service
docker compose logs -f postgres

# Stopper sans supprimer
docker compose stop

# Démarrer ce qui a été stoppé
docker compose start

# Tout arrêter, garder les données
docker compose down

# Tout arrêter ET supprimer les volumes (perte de données)
docker compose down -v

# Entrer dans un conteneur (shell)
docker exec -it labo-postgres bash

# Stats en direct
docker stats
```

Postgres en CLI

```
# Se connecter à la DB
docker exec -it labo-postgres psql -U lab_admin -d labo

# Une fois dans psql :
dt                # liste les tables
d patients        # décrit la table patients
SELECT * FROM medecins LIMIT 5;
q                # quitter
```

MongoDB en CLI

```
# Se connecter
docker exec -it labo-mongo mongosh -u lab_admin -p <motdepasse> --authenticationDatabase
admin
```

```
# Une fois dans mongosh :  
use labo  
show collections  
db.consultations.find().limit(5)  
db.consultations.countDocuments()  
exit
```

Pour conclure

Tu as maintenant les clés pour **expliquer chaque ligne de code** et **chaque choix d'architecture**. Tu sais pourquoi on a 5 conteneurs, pourquoi on mixe SQL et NoSQL, pourquoi on utilise des volumes et des healthchecks, et tu sais répondre aux questions pièges sur les secrets, les ports, les réseaux Docker.

La Session 2 va simplement ajouter *un sixième conteneur* (l'ingestor Python) qui va remplir les bases. Tous les concepts que tu viens d'apprendre s'appliquent à l'identique.